

Project Title

Password Security, Sniffing & Session Analysis

Name: Saloni Chaurasiya

Course: Cyber Security

Introduction

In this lab, Wireshark was used to capture and analyze HTTP, DNS, TCP, and ARP packets. Password auditing was performed using John the Ripper and/or Hydra to demonstrate the importance of strong passwords. The task also explores secure and insecure session management practices, including the risks of session hijacking and methods to prevent unauthorized access.

Through this practical activity, important cybersecurity concepts such as packet sniffing, authentication security, encrypted communication, and session protection mechanisms were studied and demonstrated in a safe testing environment.

Objective

The objective of this experiment is to study network security concepts by capturing and analyzing network packets using Wireshark. The task aims to understand the working of HTTP, DNS, TCP, and ARP protocols through packet sniffing and traffic analysis.

The objective of the task is to understand how network packets travel across systems, how weak passwords can be identified through auditing techniques, and how insecure session handling may expose users to security threats such as session hijacking.

Packet Capture & Traffic Analysis

Tool used: Wireshark

Start Capture: wireshark

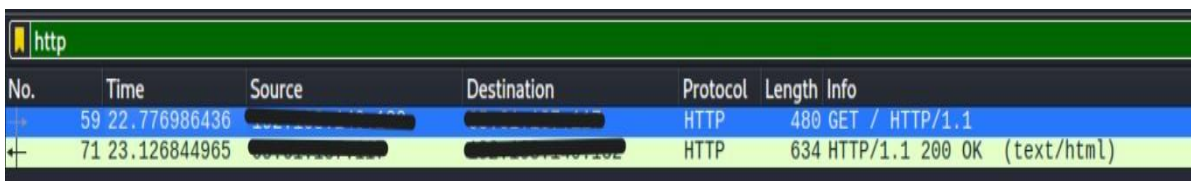
Interface (eth0 or wlan0)

Capture HTTP Traffic

Wireshark filter: http

HTTP GET request

HTTP Response 200 OK



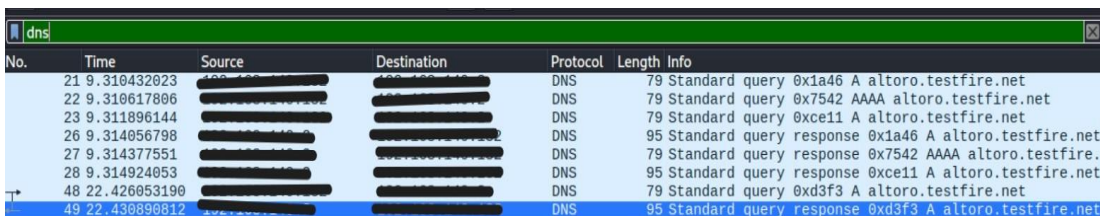
No.	Time	Source	Destination	Protocol	Length	Info
59	22.776986436	[REDACTED]	[REDACTED]	HTTP	480	GET / HTTP/1.1
71	23.126844965	[REDACTED]	[REDACTED]	HTTP	634	HTTP/1.1 200 OK (text/html)

Capture DNS Traffic

Filter: DNS

DNS Query

DNS Response



No.	Time	Source	Destination	Protocol	Length	Info
21	9.310432023	[REDACTED]	[REDACTED]	DNS	79	Standard query 0x1a46 A altoro.testfire.net
22	9.310617806	[REDACTED]	[REDACTED]	DNS	79	Standard query 0x7542 AAAA altoro.testfire.net
23	9.311896144	[REDACTED]	[REDACTED]	DNS	79	Standard query 0xce11 A altoro.testfire.net
26	9.314056798	[REDACTED]	[REDACTED]	DNS	95	Standard query response 0x1a46 A altoro.testfire.net
27	9.314377551	[REDACTED]	[REDACTED]	DNS	79	Standard query response 0x7542 AAAA altoro.testfire.net
28	9.314924053	[REDACTED]	[REDACTED]	DNS	95	Standard query response 0xce11 A altoro.testfire.net
48	22.426053190	[REDACTED]	[REDACTED]	DNS	79	Standard query 0xd3f3 A altoro.testfire.net
49	22.430890812	[REDACTED]	[REDACTED]	DNS	95	Standard query response 0xd3f3 A altoro.testfire.net

Capture TCP Handshake

Filter: TCP

TCP 3-way handshake

- SYN
- SYN ACK
- ACK

Secure vs Insecure Session Management

Insecure Session Examples

- Session ID in URL:

`http://site.com/profile.php?sessionid=12345`

demonstrates a **session ID passed in the URL**.

This is considered insecure session management because:

- session IDs become visible in:
 - browser history
 - logs
 - screenshots
 - bookmarks
 - shared links
- attackers may steal the session token

This can lead to:

- session hijacking

- unauthorized account access

Insecure Method

http://site.com/profile.php?sessionid=12345

Risk:

- Session token exposed in URL
-

Secure Method

Modern websites usually use:

- HTTPS
- secure cookies
- HttpOnly cookies
- SameSite protection

Example:

Set-Cookie: PHPSESSID=abc123; Secure; HttpOnly

Session Hijacking Risks

Cookie: PHPSESSID=abcd1234

This is an example of a **session cookie**.

Meaning:

- PHPSESSID → session identifier used by PHP applications

- abcd1234 → unique session token assigned to the user

Unlike putting session IDs in URLs, storing them in cookies is the standard and more secure approach.

Risk

If attackers steal the session cookie:

- they may impersonate the logged-in user
- access the account without password

Common causes:

- XSS attacks
- insecure HTTP
- malware/sniffing
- weak session management

Session cookies such as PHPSESSID are commonly used for authentication. Secure cookie attributes like Secure, HttpOnly, and SameSite help reduce session hijacking risks.

Prevention Techniques

- Use HTTPS everywhere
- Regenerate session after login
- Set Secure + HttpOnly cookie
- SameSite cookies
- Logout destroys token

- MFA login

Conclusion

This experiment demonstrated that network sniffing can reveal sensitive metadata, weak passwords are vulnerable to dictionary attacks, and insecure session management may allow account compromise. Strong passwords, encrypted traffic, and secure session controls significantly reduce risk.